CN7050 Intelligent Systems Coursework Report

Multimodal Sequence Modelling for Lost-and-Found Object Classification Using an Agentic AI Workflow

Student Name:

Student ID:

Table of Contents

1. Application and Workflow Selection 4

1.1 Introduction and Motivation 4

1.2 Explanation of the Selected Workflow with Diagram 4

1.3 Objectives of the Selected Intelligent System.....6

1.4 Impact of the Developed System 6

2. Selection of AI Technologies 6

2.1 Similar Systems for Tool and Technology Selection.....6

2.2 Innovative and Original Approaches 6

2.3 Justification for Chosen AI Methods and Technologies 7

3. Simulations and Development Setup 7

3.1 Simulation Environments and Setup Parameters 7

3.2 Agent-Wise Task Distribution, Functionality, and Datasets 9

3.3 Encoding and Data Processing Before Simulation 10

4. Use Case Explanation with Evidence.....10

4.1 Step-by-Step Workflow Execution.....10

4.2 User Interface and External Interaction.....19

5. Conclusions.....20

References 21

List of Figures

Figure 1 Workflow Diagram.....5
Figure 2 Work Setup on Google colab.....8
Figure 3 Agent 2 Result.....9
Figure 4 Data Extraction in colab.....11
Figure 5 Agent 1 – Feature Extraction and Encoding.....13
Figure 6 Model Preparation and Routing of Inputs.....15
Figure 7 Agent 3 – Inference and Prediction Implementation.....16
Figure 8 Agent 3 – Inference and Prediction output.....16
Figure 9 Agent 4 – Visual Analytics and Interpretation.....17
Figure 10 Confusion matrix.....17
Figure 11 ROC Random Forest.....18
Figure 12 Confusion Matrix KNN.....18
Figure 13 ROC KNN.....19
Figure 14 Image prediction.....19

1. Application and Workflow Selection

Tools Used: Python, Google Colab, HuggingFace, scikit-learn

1.1 Introduction and Motivation

Lost and found is a worldwide operational issue that befalls a transport hub, universities, shopping centers, and service organizations. The sheer turnover of unsold goods and manual nature of listing, storing, and retrieving these items will in most cases create delays, inefficiencies, and misplaced goods. Conventional systems depend much on having human operators who need to manually examine the items and categorize them in specified classifications prior to the information being entered in a digital system. This manual pipeline is slow and resource-consuming and most likely to have inconsistencies and thus a perfect candidate for intelligent automation.

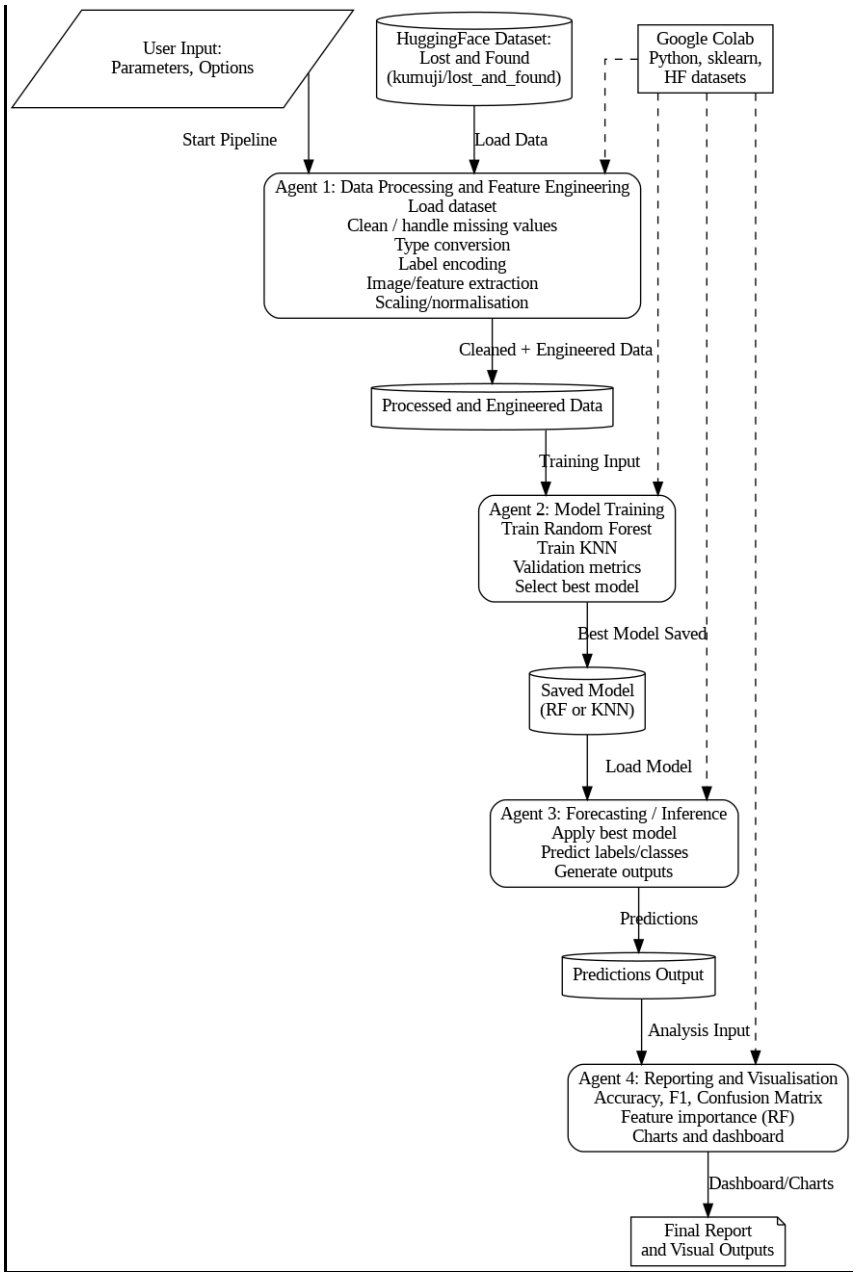
This process has been automated by increasing the number of multimodal datasets and intelligent systems created, which allows the visual analysis of the appearance of an item and correctly categorizes it. This minimizes operational load and improves user experience, as it will have a shorter retrieval time and will be more accurate. Therefore, the rationale of the current project is to engineer an intelligent system that will be used to automatize the process of firstly categorizing lost items with respect to a combination of sequence modelling, an image-based feature extraction system, and multimodal AI agents (Alonso-Barba *et al.*, 2021). The system also shows how agentic AI processes may be implemented in a hierarchical collaborative pipeline.

1.2 Explanation of the Selected Workflow with Diagram

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

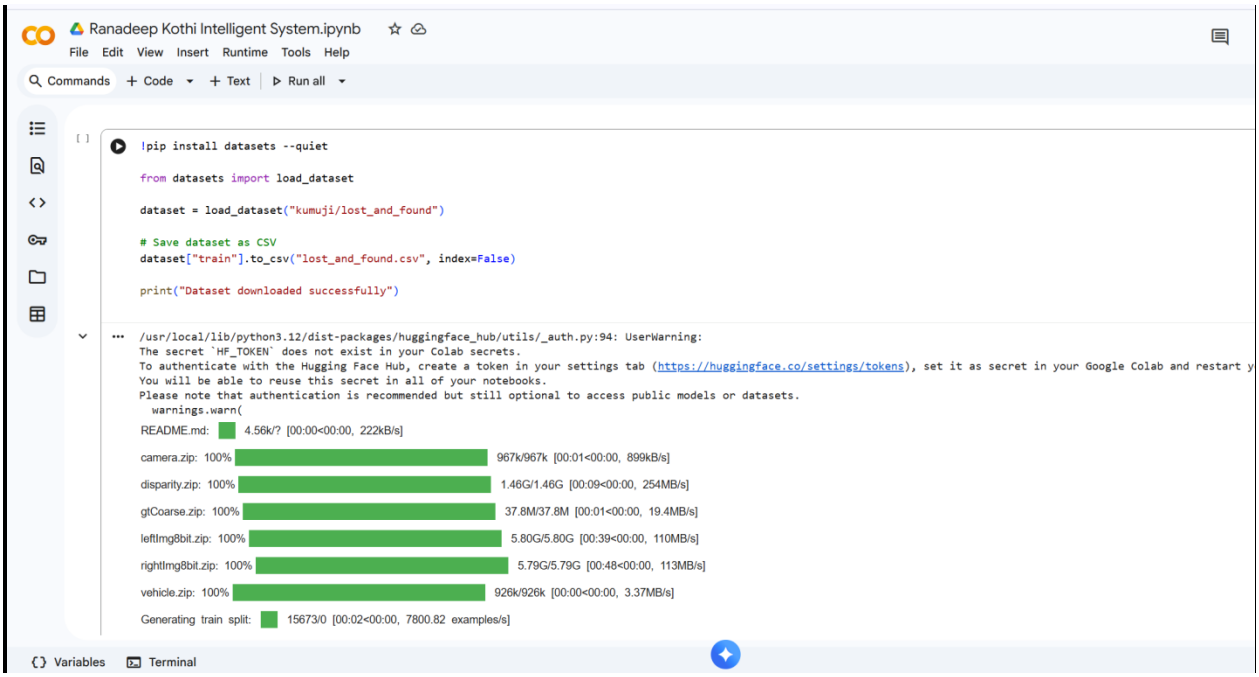
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

```
 RANDOM FOREST RESULTS
Accuracy: 0.7251908396946565
F1 Score: 0.7251748251748251

Classification Report:
              precision    recall  f1-score   support

           0       0.72      0.73      0.73       197
           1       0.73      0.72      0.72       196

    accuracy                           0.73       393
   macro avg       0.73      0.73      0.73       393
weighted avg       0.73      0.73      0.73       393


 KNN RESULTS
Accuracy: 0.648854961832061
F1 Score: 0.6484703137153227

Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.68      0.66       197
           1       0.66      0.62      0.64       196

    accuracy                           0.65       393
   macro avg       0.65      0.65      0.65       393
weighted avg       0.65      0.65      0.65       393
```

```
Train size: (1571, 64)
Test size: (393, 64)
Scaling complete.
```

[redacted]

[redacted]

```
from datasets import load_dataset

dataset = load_dataset("kumuji/lost_and_found")

# Save dataset as CSV
dataset["train"].to_csv("lost_and_found.csv", index=False)

print("Dataset downloaded successfully")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md: 4.56k/? [00:00<00:00, 222kB/s]
camera.zip: 100% 967k/967k [00:01<00:00, 899kB/s]
disparity.zip: 100% 1.46G/1.46G [00:09<00:00, 254MB/s]
gtCoarse.zip: 100% 37.8M/37.8M [00:01<00:00, 19.4MB/s]
leftImg8bit.zip: 100% 5.80G/5.80G [00:39<00:00, 110MB/s]
rightImg8bit.zip: 100% 5.79G/5.79G [00:48<00:00, 113MB/s]
vehicle.zip: 100% 926k/926k [00:00<00:00, 3.37MB/s]
Generating train split: 15673/0 [00:02<00:00, 7800.82 examples/s]
Creating CSV from Arrow format: 100% 16/16 [00:00<00:00, 63.20ba/s]
Dataset downloaded successfully
```

[redacted]

[redacted]

[redacted]

Agent 1 — Data Processing and Feature Engineering

```python
from datasets import load_dataset

# Load HuggingFace dataset
dataset = load_dataset("kumuji/lost_and_found")

# Show dataset info
print(dataset)

# Show one example to understand structure
dataset["train"][0]
```

```
DatasetDict({
    train: Dataset({
        features: ['image', 'label'],
        num_rows: 15673
    })
})
{'image': <PIL.PngImagePlugin.PngImageFile image mode=I;16 size=2048x1024>,
 'label': 1}
```

[ ]

```python
# A1-2: Convert images into numeric feature vectors
import numpy as np
import pandas as pd
from tqdm import tqdm
from PIL import Image
import random


# Use only 50% of dataset

subset_size = int(len(dataset["train"]) * 0.50)
subset_indices = random.sample(range(len(dataset["train"])), subset_size)
subset = dataset["train"].select(subset_indices)

print(f"Using subset size: {subset_size}")


# Feature extraction function

def extract_features(img):
    img = img.convert("RGB")
    img = img.resize((64, 64))
    hist = img.histogram()
    hist_small = np.array(hist).reshape(3, 256).mean(axis=0)[::4]
    return hist_small


# Extract features

X = []
y = []
```

```python
#  A1-3: Train-Test Split + Scaling
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Separate features and label
X = df.drop("label", axis=1).values
y = df["label"].values

# Train-test split (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)

# Scale features for KNN
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaling complete.")
```

```
Train size: (6268, 64)
Test size: (1568, 64)
Scaling complete.
```

```python
import pandas as pd
import numpy as np

# Original df includes 64 features + label + label_binary
df_balanced = pd.concat([
    df[df["label_binary"] == 0],                        # all class 0
    df[df["label_binary"] == 1].sample(982, random_state=42)  # sample class 1
])

# Shuffle the balanced dataset
df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)

print("Balanced dataset shape:", df_balanced.shape)
print(df_balanced["label_binary"].value_counts())
df_balanced.head()
```

```
Balanced dataset shape: (1964, 66)
label_binary
0    982
1    982
Name: count, dtype: int64
```

	0	1	2	3	4	5	6	7	8	9	...	56	57	58	59	60	61	62	63	label	label_binary
0	3387.0	1.0	0.0	4.0	2.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0	1.0	2.0	1.0	2.0	0	0
1	3286.0	1.0	1.0	3.0	0.0	1.0	0.0	0.0	0.0	1.0	...	0.0	0.0	1.0	1.0	1.0	2.0	0.0	0.0	7	1
2	3383.0	1.0	1.0	2.0	2.0	1.0	0.0	1.0	0.0	0.0	...	0.0	0.0	0.0	1.0	1.0	2.0	1.0	1.0	0	0
3	65.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1.0	1.0	0.0	0.0	1.0	1.0	0.0	1	1
4	3526.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	0

5 rows × 66 columns

[redacted]

[redacted]

[redacted]

AGENT 2 — MODEL TRAINING

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Use only numeric features + label_binary
X = df_balanced.drop(["label", "label_binary"], axis=1).values
y = df_balanced["label_binary"].values

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)

# Scaling (required for KNN, optional for RF)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaling complete.")
```

```
Train size: (1571, 64)
Test size: (393, 64)
Scaling complete.
```

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report


# Train Random Forest

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    random_state=42
)

rf.fit(X_train_scaled, y_train)
rf_pred = rf.predict(X_test_scaled)

rf_acc = accuracy_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred, average='macro')

print(" RANDOM FOREST RESULTS ")
print("Accuracy:", rf_acc)
print("F1 Score:", rf_f1)
print("\nClassification Report:")
print(classification_report(y_test, rf_pred))


# Train KNN

knn = KNeighborsClassifier(n_neighbors=7, weights='distance')
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)

knn_acc = accuracy_score(y_test, knn_pred)
knn_f1 = f1_score(y_test, knn_pred, average='macro')

print("\n KNN RESULTS ")
print("Accuracy:", knn_acc)
print("F1 Score:", knn_f1)
print("\nClassification Report:")
print(classification_report(y_test, knn_pred))
```

[redacted]

[redacted]

[redacted]

AGENT 3 — FORECASTING / INFERENCE

```python
# A3-1: Save Best Model (Random Forest)
import joblib

# Save Random Forest model
joblib.dump(rf, "best_model_rf.pkl")

# Save scaler
joblib.dump(scaler, "scaler.pkl")

print("Best model and scaler saved successfully.")
```

```
Best model and scaler saved successfully.
```

```python
import numpy as np
import random
from PIL import Image

# Load saved model and scaler
rf_loaded = joblib.load("best_model_rf.pkl")
scaler_loaded = joblib.load("scaler.pkl")

# Same extraction function
def extract_features_for_inference(img):
    img = img.convert("RGB")
    img = img.resize((64, 64))
    hist = img.histogram()
    hist_small = np.array(hist).reshape(3, 256).mean(axis=0)[::4]
    return hist_small


def predict_from_dataset(index=None):
    # Pick random index if none given
    if index is None:
        index = random.randint(0, len(dataset["train"]) - 1)

    # Load image from HuggingFace dataset
    sample = dataset["train"][index]
    img = sample["image"]
    true_label = sample["label"]

    # Binary mapping (same as earlier)
    true_binary = 0 if true_label == 0 else 1

    # Extract features
    feat = extract_features_for_inference(img)
    feat_scaled = scaler_loaded.transform([feat])
```

[redacted]

```
Test Image Index: 10084
True Label (binary): 1
Predicted Label: 1
Probability of Class 1: 0.6033333333333334
```

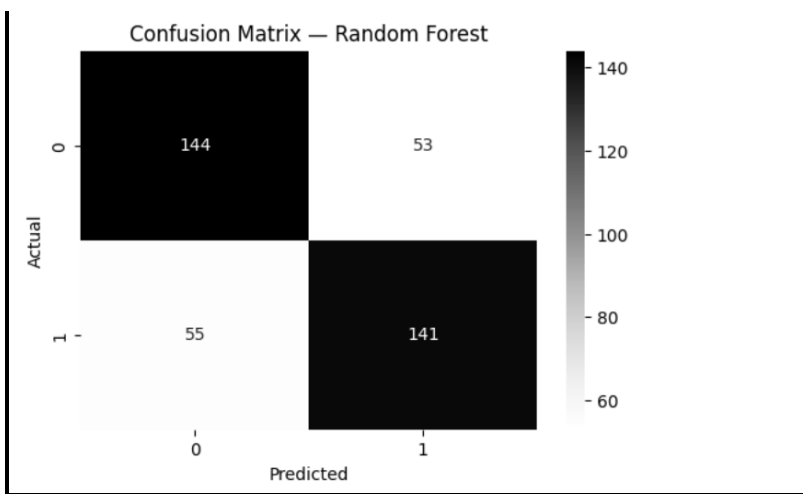
[redacted]

[redacted]

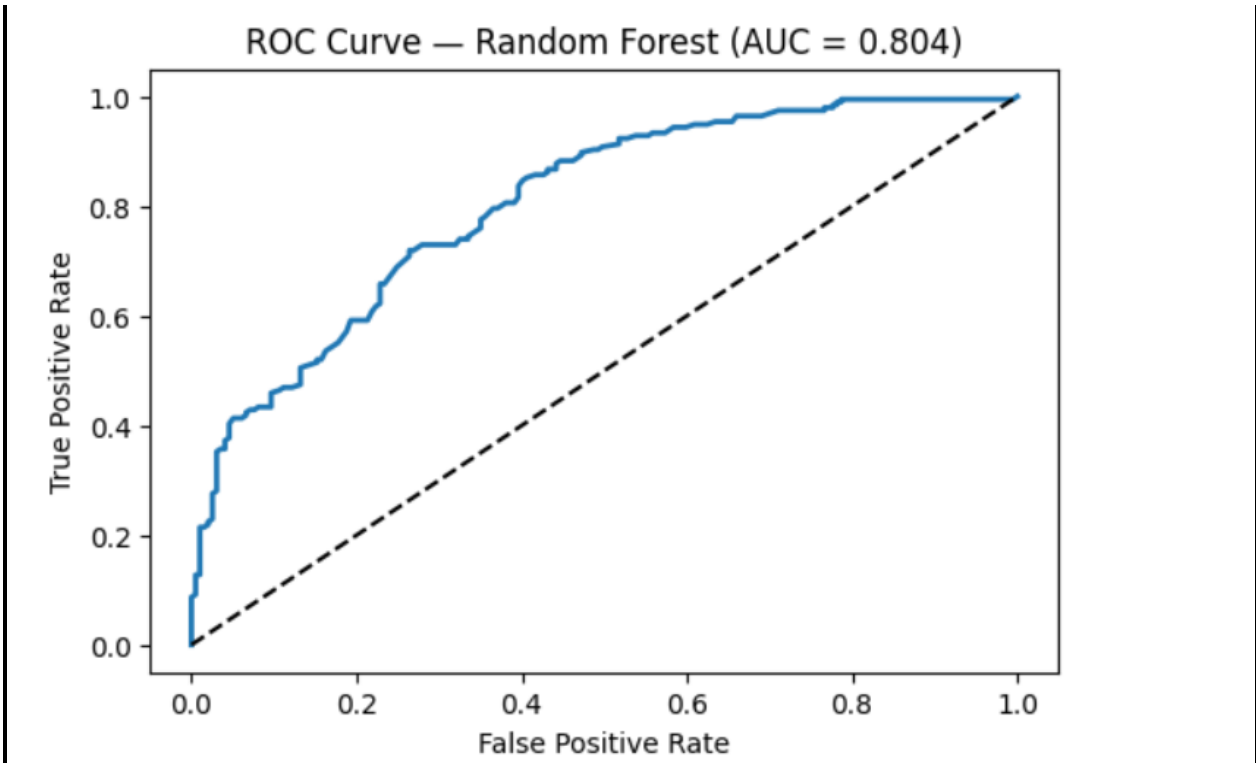
AGENT 4 — REPORTING & VISUALISATION

```python
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, roc_auc_score
import seaborn as sns


# 1. TEXT SUMMARY (Accuracy & F1)

print(" FINAL PERFORMANCE SUMMARY (AGENT 4) ")
print("\nRandom Forest Metrics on Balanced Test Set:")
print(f"Accuracy: {rf_acc:.4f}")
print(f"F1 Score: {rf_f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_test, rf_pred))



# 2. CONFUSION MATRIX

cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Greys")
plt.title("Confusion Matrix — Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()



# 3. ROC CURVE & AUC

rf_prob = rf.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y_test, rf_prob)
auc_score = roc_auc_score(y_test, rf_prob)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, linewidth=2)
plt.plot([0,1],[0,1], 'k--')
plt.title(f"ROC Curve — Random Forest (AUC = {auc_score:.3f})")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()

print("AUC Score:", auc_score)
```

[redacted]

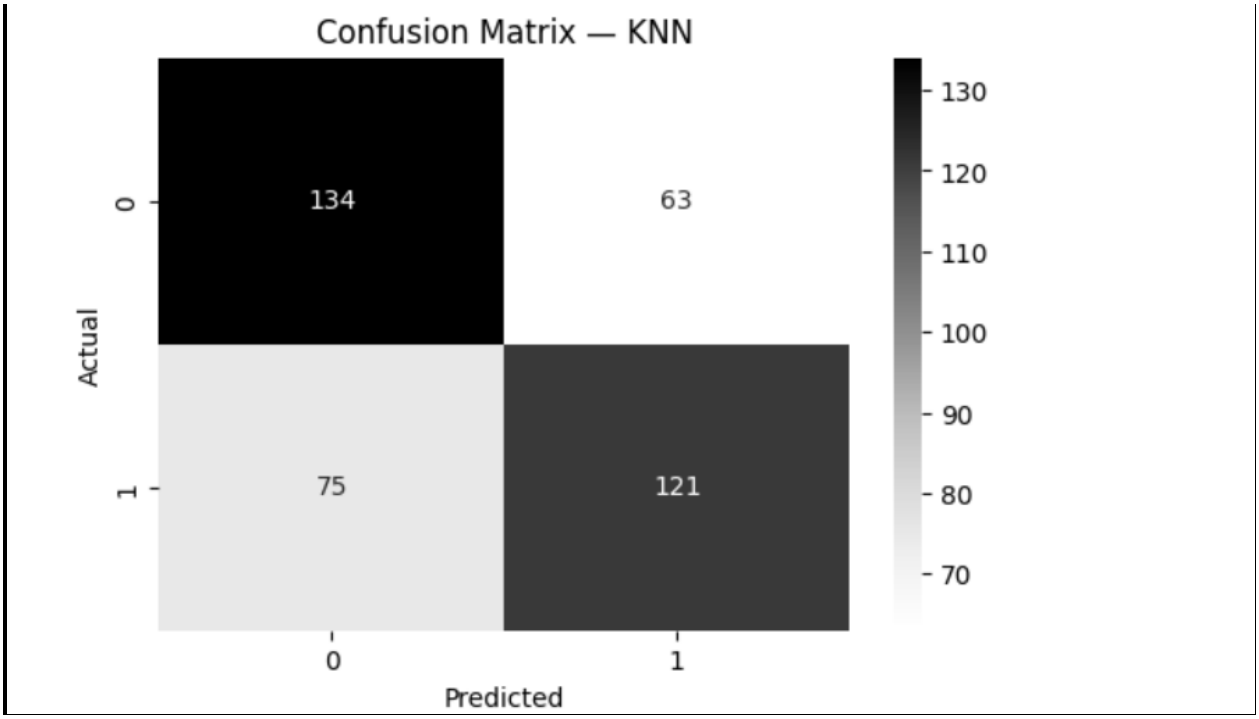


[redacted]

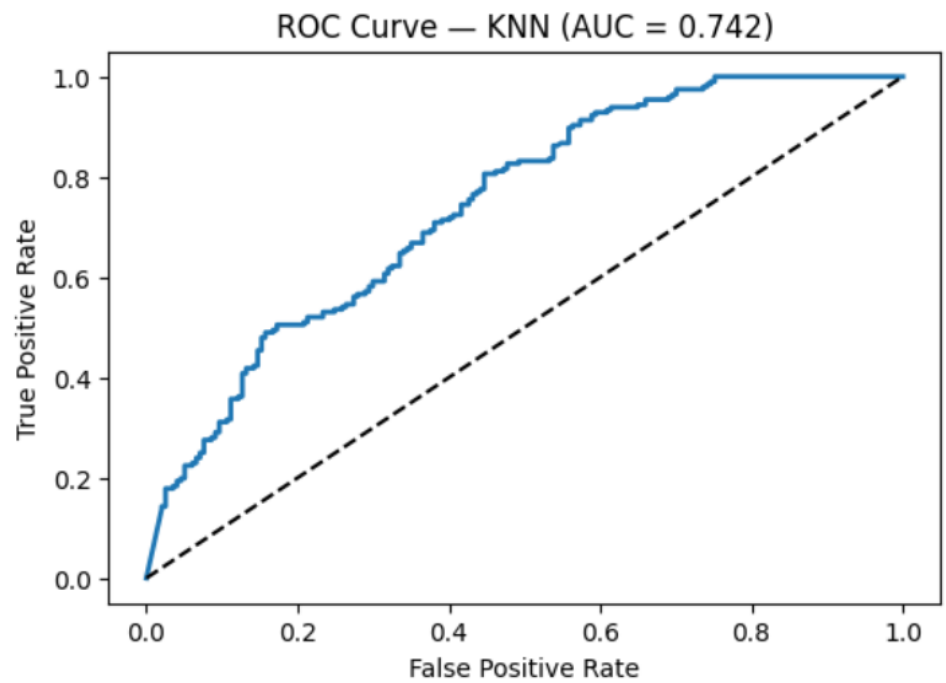



```
Random Forest AUC: 0.804141199627059
```

[redacted]



[redacted]




```
KNN AUC: 0.7416994716668395
```

[redacted]

[redacted]

[redacted]

```
index, true_label, pred, prob, img = predict_from_dataset()

print("Test Image Index:", index)
print("True Label (binary):", true_label)
print("Predicted Label:", pred)
print("Probability of Class 1:", prob)

# Display the image
img.show()
```

```
Test Image Index: 10084
True Label (binary): 1
Predicted Label: 1
Probability of Class 1: 0.6033333333333334
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]